1.0 Introduction

The “Kakak” app aims to meet the increasing need for effective and trustworthy household management options. In the modern, rapid-moving world, numerous families encounter difficulties in juggling work, family, and home duties. The “Kakak” application links users to skilled domestic workers to assist with crucial responsibilities such as cleaning, childcare, and elderly support, providing a smooth and hassle-free solution. The app offers a platform that allows customers to effortlessly hire skilled workers according to their individual requirements, ensuring a customized experience. By incorporating features such as background checks, reviews, and an intuitive interface, “Kakak” promotes trust and transparency among users and workers. In addition to standard cleaning services, “Kakak” provides specialized options like childcare, assisting families in handling their daily activities effortlessly. Although there are challenges like worker availability and consistency in service, the app’s all-encompassing method provides a dependable, affordable, and convenient solution for contemporary household management, while simultaneously generating income opportunities for domestic workers. “Kakak” is the premier platform for a more intelligent and manageable household.

2.0 Problem Solving and Design

2.1 Functionalities

Kakak housekeeping app provide the following function to solve user problem

- Customers and workers can register a new account by providing necessary details such as email and passwords.
- Customers, workers and admin can log in to existing accounts using their emails and credentials.
- Customers, workers and admin can log out from the account to end the session.
- Customers, workers and admin can modify their default settings in the application.
- Customers, workers and admin can manage their profile by updating name, password, profile picture, and contact number.
- Customers can choose services (e.g., household chores, childcare) for booking.
- Customers can browse and select workers based on name and availability.
- Customers can make bookings and choose payment methods (cash, e-wallet, online banking).
- Customers can edit the time schedule before workers accept the jobs.
- Customers can leave reviews for workers after service completion.
- Customers can contact admin for assistance or inquiries.
- Customers can view their booking history and manage past bookings.
- Customers can manage their account by updating name, password, profile picture, and contact details.
- Workers can view and update their schedule, including marking unavailable times.
- Workers can see reviews and ratings left by customers.
- Workers can view their total income through the app.
- Admins can manage customer and worker accounts (create, update, or deactivate).
- Admins can manage platform content, such as service descriptions, FAQs, and terms and conditions.
- Admins can monitor worker performance by tracking ratings and feedback.

- Admins can handle and resolve refund-related issues.
- Admins can review and resolve complaints from customers and workers.
- Admins can generate reports and analytics on business performance and operations.

2.2 Use Case Diagram

Figure 1: Use Case Diagram of Kakak App

2.4 Dynamic Data Management

```
Future<void> _loadUserId() async {
  User? currentUser = FirebaseAuth.instance.currentUser;
  if (currentUser != null) {
    setState(() {
      userId = currentUser.uid;
    });
  } else {
    print('No user is logged in.');
  }
}
```

Figure 3: Function _loadUserId

This function dynamically retrieves the current UID from Firebase Authentication. The data is subject to change based on whether a user is logged in or out, making it dynamic.

```
@override
Widget build(BuildContext context) {
  return StreamBuilder<QuerySnapshot>(
    stream: FirebaseFirestore.instance
        .collection('bookings')
        .where('workerId', isEqualTo: currentUser?.uid)
        .where('status', isEqualTo: 'pending')      You, 2 days ago • 123
        .snapshots(),
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Center(child: CircularProgressIndicator());
      }
```

Figure 4: Fetching Bookings for Task List

The StreamBuilder connects to Firebase Firestore to listen for updates in the bookings collection. This dynamic data stream helps ensure that the task list is always up to date with the latest booking information, making the application responsive to real-time changes.

```
Future<void> acceptTask(Map<String, dynamic> booking, int index) async {
  try {
    if (currentUser != null) {
      final taskData = {
        'workerId': currentUser!.uid,
        'serviceType': booking['serviceType'],
        'serviceTime': booking['serviceDate'],
        'serviceLocation': booking['serviceLocation'],
        'status': 'In Progress',
        'bookingId': booking['id'], // Adding bookingId to taskData
      };

      await FirebaseFirestore.instance
          .collection('bookings')
          .doc(booking['id'])        You, 2 days ago • 123
          .update({'status': 'accepted'});

      final taskRef = await FirebaseFirestore.instance.collection('worker selected task').add(taskData);
      taskData['id'] = taskRef.id;
      SelectedTaskListState.addSelectedTask(taskData);
```

```
Future<void> rejectTask(Map<String, dynamic> booking, int index) async {
  try {
    await FirebaseFirestore.instance.collection('bookings').doc(booking['id']).delete();

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Task rejected.')),
    );
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error deleting task: $e')),
    );
  }
}
```

Figure 5: Accept and reject task function

Accepting and rejecting tasks dynamically updates the status of bookings in the database. The acceptTask function changes the task status to “accept” and adds t to the “worker selected task” collection, while rejectTask deletes the booking. These functions invovle dynamic data management since they alter and persist changes directly in the Firestore database.

```
final snapshot = await FirebaseFirestore.instance
    .collection('worker selected task')
    .where('workerId', isEqualTo: currentUser!.uid)
    .get();
setState(() {
  selectedTasks = snapshot.docs.map((doc) {
    final data = doc.data();
    data['id'] = doc.id; // Add the document ID to each task
    return data;
  }).toList();
});
}
```

Figure 6: Fetching Selected Tasks

This method fetches the tasks that have benn selected by the worker. Data is dynamically retrieved and used to populate the UI. The use of setState() ensures that the UI reflects the current data in real time.

```
child: StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance
      .collection('reviews')
      .where('workerId', isEqualTo: currentUser.uid)
      .snapshots(),
  builder: (context, snapshot) {
```

Figure 7: Fetching Reviews (StreamBuilder<QuerySnapshot>)

The StreamBuilder listens to changes in the'reviews' collection in Firestore.

```
body: StreamBuilder<QuerySnapshot>(
  stream: _firestore
      .collection('bookings')
      .where('status', isEqualTo: 'completed')
      .where('customerId', isEqualTo: _auth.currentUser!.uid)
      .snapshots(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return Center(child: CircularProgressIndicator());
    } else if (snapshot.hasError) {
      return Center(child: Text("Error loading bookings"));
    } else if (!snapshot.hasData || snapshot.data!.docs.isEmpty) {
      return Center(child: Text("No completed bookings available for review"));
    } else {
      final completedBookings = snapshot.data!.docs;

      return ListView.builder(
        itemCount: completedBookings.length,
        itemBuilder: (context, index) {
          final booking = completedBookings[index].data() as Map<String, dynamic>;
          final workerId = booking['workerId'];
```

Figure 8: Using StreamBuilder

Using StreamBuilder to listen to the changes in the booking data in real-time. It listens from collection "bookings" and filter by "userId".

```
WorkerDetailsPage({required this.workerId, required this.selectedDate, required this.serviceType});

// Function to retrieve worker details from Firestore
Future<Map<String, dynamic>?> getWorkerDetails() async {
  try {
    DocumentSnapshot doc = await FirebaseFirestore.instance.collection('Worker Profile').doc(workerId).get();
    return doc.exists ? doc.data() as Map<String, dynamic> : null;
  } catch (e) {
    print('Error fetching worker details: $e');
    return null;
  }
}
```

Figure 9: getWorkerDetails function

This function is used FutureBuilder to fetch specific worker data from “Worker Profile” collection based n workerId.

```
// StreamBuilder to retrieve reviews
StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance
      .collection('reviews')
      .where('workerId', isEqualTo: workerId)
      .snapshots(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return Center(child: CircularProgressIndicator());
    }
    if (snapshot.hasError) {
      return Center(child: Text('Error loading reviews.'));
    }
    if (!snapshot.hasData || snapshot.data!.docs.isEmpty) {
      return Text('No reviews available.');
    }

    final reviews = snapshot.data!.docs;

    return Column(
      children: reviews.map((doc) {
        final reviewData = doc.data() as Map<String, dynamic>;
        final reviewerName = reviewData['customerName'] ?? 'Anonymous';
        final reviewText = reviewData['reviewText'] ?? 'No review text.';
        final rating = reviewData['rating'] ?? 0;

        return ReviewTile(
          reviewerName: reviewerName,
          reviewText: reviewText,
          rating: rating,
        ); // ReviewTile
      }).toList(),
    ); // Column
  },
), // StreamBuilder
```

Figure 10: StreamBuilder to retrieve review

StreamBuilder listens to the real-time changes in “reviews” collection which is filter by workerId to load the reviews based on the specific worker.

```
// Start Booking Button
Center(
  child: ElevatedButton(
    onPressed: () {
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => BookingPage(
            workerId: workerId,
            selectedDate: selectedDate,
            serviceType: serviceType,
          ), // BookingPage
        ), // MaterialPageRoute
      );
    },
```

Figure 11: Navigate to booking page with workerId, seletedDate and serviceType

“Start Booking” button navigate to BookingPage with parameter workerId, selectedDate and serviceType.

2.5 CRUD

2.5.1 Create

```
class UserRepository {
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;

  // Save a UserWorker to Firestore
  Future<void> createUserWorker(String id, UserWorker userWorker) async {
    await _firestore.collection('users').doc(id).set(userWorker.toMap());
  }

  // Save a UserCustomer to Firestore
  Future<void> createUserCustomer(String id, UserCustomer userCustomer) async {
    await _firestore.collection('users').doc(id).set(userCustomer.toMap());
  }
}
```

Figure 12: createUserWorker and createUserCustomer Function

This function createUserWorker and createUserCustomer creates or updates documents in the "users" collection for UserWorker and UserCustomer types, using each class's toMap() method to convert the objects into key-value pairs suitable for Firestore storage.

```
gnment > lib > screen_worker > worker_home_screen.dart > _TaskListState > acceptTask
class _TaskListState extends State<TaskList> {
  Future<void> acceptTask(Map<String, dynamic> booking, int index) async {
    try {
      if (currentUser != null) {
        final Map<String, dynamic> taskData = <String, dynamic>{
          'workerId': currentUser!.uid,
          'serviceType': booking['serviceType'],
          'serviceTime': booking['serviceDate'],
          'serviceLocation': booking['serviceLocation'],
          'status': 'In Progress',
          'bookingId': booking['id'], // Adding bookingId to taskData
        };

        await FirebaseFirestore.instance
            .collection(collectionPath: 'bookings')
            .doc(path: booking['id'])
            .update(data: <Object, Object?>{'status': 'accepted'});

        final DocumentReference<Map<String, dynamic>> taskRef = await FirebaseFirestore.instance.collection(coll… 'worker selected task').add(task
        taskData['id'] = taskRef.id;
        SelectedTaskListState.addSelectedTask(taskData: taskData);

        // Update unavailableDates in worker profile
        await FirebaseFirestore.instance
            .collection(collectionPath: 'Worker Profile')      You, 46 minutes ago • 123
            .doc(path: currentUser!.uid)
            .update(data: <Object, Object?>{
          'unavailableDates': FieldValue.arrayUnion(elements: <dynamic>[booking['serviceDate']]),
        });

        ScaffoldMessenger.of(context: context).showSnackBar(
          snackBar: const SnackBar(content: Text(data: 'Task accepted and added to selected tasks!')),
```

Figure 13: Accept task function

This function is to create a collection named 'worker selected task' to store specific informations in the specific fields.

```
// CREATE: Add a new worker profile
Future<void> addWorkerProfile(Worker worker) async {
// Use the UID as the document ID
await FirebaseFirestore.instance
    .collection('Worker Profile')
    .doc(worker.id) // Ensure this is the UID from the current user        You, 2 days ago
    .set(worker.toMap(), SetOptions(merge: true)); // Use merge to update if it exists
```

Figure 14: _addWorkerProfile

This function helps to create a new collection named 'Worker Profile' in Firebase.

```
// Save profile data using the repository
Future<void> _saveProfile() async {
  if (formKey.currentState!.validate()) {
    try {
      await _saveDataLocally();

      // Parse the unavailable date string into a DateTime object
      DateTime? unavailableDate;
      if (unavailableDateController.text.isNotEmpty) {
        unavailableDate = _parseDate(unavailableDateController.text);
      }

      Worker worker = Worker(
        id: userId, // Use the UID from the current user
        workerName: usernameController.text,
        contact: contactController.text,
        profileDetails: profileDetailsController.text,
        location: locationController.text,
        ratings: 0.0,
        totalIncome: 0.0,
        unavailableDates: unavailableDate != null ? [unavailableDateController.text] // Store as a String
        : [],
      );

      await _workerRepository.addWorkerProfile(worker); // Use repository method

      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Profile saved successfully!')),
      );
```

Figure 15: _saveProfile function

This function creates a new worker profile and adds it to Firestore using the Worker Repository's addWorkerProfile method and stores specific information in specific fields.

```dart
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:mae_assignment/booking.dart';

class BookingRepository {
  final CollectionReference _bookingsCollection =
      FirebaseFirestore.instance.collection('bookings');

  /// Adds a new booking to the Firestore collection.
  /// Throws an error if the booking could not added.
  Future<void> addBooking(Booking booking) async {
    try {
      // Add the booking and Firestore will generate the document ID
      DocumentReference docRef = await _bookingsCollection.add(booking.toMap());

      // Update the booking object with the Firestore generated ID
      booking.bookingId = docRef.id;

      print("Booking added with ID: ${docRef.id}");
    } catch (e) {
      print('Error adding booking: $e');
      throw Exception('Failed to add booking');
    }
  }
}
```

Figure 16: Booking Repository

This code creates or updates documents in the “bookings” collection, using booking class’s toMap() method to convert the objects into key-value pairs suitable for Firestore storage.

```dart
class ReviewRepository {
  final FirebaseFirestore _firestore;

  ReviewRepository({FirebaseFirestore? firestore}) : _firestore = firestore ?? FirebaseFirestore.instance;

  Future<void> addReview(Review review) async {
    await _firestore.collection('reviews').add(review.toMap());
  }
}
```

Figure 17: Review Repository

This code creates or updates documents in the “reviews” collection, using booking class’s toMap() method to convert the objects into key-value pairs suitable for Firestore storage.

2.5.2 Read

```
_buildBookingDetail("Date", bookingData["serviceDate"]),
_buildBookingDetail("Service Type", bookingData["serviceType"]),
_buildBookingDetail("Location", bookingData["serviceLocation"]),
_buildBookingDetail("Status", bookingData["status"]),
FutureBuilder<DocumentSnapshot>(
  future: _firestore.collection('Worker Profile').doc(bookingData['workerId']).get(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return _buildBookingDetail("Worker", "Loading...");
    } else if (snapshot.hasError || !snapshot.hasData || !snapshot.data!.exists) {
      return _buildBookingDetail("Worker", "Unknown");
    } else {
      final workerName = snapshot.data!['workerName'];
      return _buildBookingDetail("Worker", workerName ?? "Unknown");
    }
  },
), // FutureBuilder
```

Figure 18: bookingData function

The bookingData contains booking details like serviceDate, serviceType, serviceLocation, and status. Reads the worker's profile from the Firestore collection "Worker Profile" using the workerId stored in bookingData.

```
Future<void> _loadUserData() async {
  User? currentUser = FirebaseAuth.instance.currentUser;
  if (currentUser != null) {
    DocumentSnapshot userDoc = await FirebaseFirestore.instance
        .collection('users')
        .doc(currentUser.uid)
        .get();

    setState(() {
      userName = userDoc['name'] ?? 'No name'; // Display name by default
    });
  }
}
```

Figure 19: _loadUserData function

This code use to read from "users" collection to load name.

```
Future<void> fetchSelectedTasks() async {
  if (currentUser == null) return;

  final QuerySnapshot<Map<String, dynamic>> snapshot = await FirebaseFirestore.instance
      .collection(collectionPath: 'worker selected task')
      .where(field: 'workerId', isEqualTo: currentUser!.uid)
      .get();
  setState(fn: () {
    selectedTasks = snapshot.docs.map<Map<String, dynamic>>(toElement: (QueryDocume… doc) {
      final Map<String, dynamic> data = doc.data();
      data['id'] = doc.id; // Add the document ID to each task
      return data;
    }).toList();
  });
}
```

Figure 20: fetchSelectedTasks function

This function fetches the list of selected tasks by reading the workerId as current UID from the worker selected task collection for the current worker.

```
Future<void> _loadUsername() async {
  if (userId.isEmpty) return;
  try {
    DocumentSnapshot userDoc = await FirebaseFirestore.instance
        .collection('users')
        .doc(userId)
        .get();

    if (userDoc.exists) {
      setState(() {
        username = userDoc['name'] ?? 'No name available';
      });
    } else {
      print('User document not found');
      setState(() {
        username = 'User not found';
      });
    }
  } catch (e) {
    print('Error fetching username: $e');
    setState(() {
      username = 'Error fetching username';
    });
  }
}
```

Figure 21: _loadUsername function

This function retrieves the username from “users” collection by reading the document id as current UID.

```
child: StreamBuilder<QuerySnapshot>(
  stream: FirebaseFirestore.instance.collection('reviews').snapshots(),
  builder: (context, snapshot){
    if(snapshot.connectionState == ConnectionState.waiting){
      return const Center(child: CircularProgressIndicator());
    }

    if(!snapshot.hasData || snapshot.data!.docs.isEmpty){
      return const Center(
        child: Text(
          'No reviews available.',
          style: TextStyle(color: Colors.grey, fontSize: 16),
        ), // Text
      ); // Center
    }
    final reviews = snapshot.data!.docs;
```

Figure 22: StreamBuilder in ReviewRatingScreen

The StreamBuilder listens for real-time updates from the 'reviews' collection in Firestore. It listens for any new or updated documents and displays them in the UI.

```
Future<void> fetchEarnings() async {
  if (currentUser == null) return;

  try {
    // Retrieve earnings data from Firestore
    final snapshot = await FirebaseFirestore.instance
        .collection('worker selected task')
        .where('workerId', isEqualTo: currentUser!.uid)
        .get();

    final List<Map<String, dynamic>> earningsList = snapshot.docs.map((doc) {
      final data = doc.data();
      data['id'] = doc.id;
      return data;
    }).toList();

    setState(() {
      earnings = earningsList;
      totalEarnings = earnings.fold(0, (sum, earning) => sum + (earning['amount'] as int? ?? 100));
    });

    //Store total earnings i Firebase
    await storeTotalEarnings();
  } catch (e) {
    print('Error fetching earnings data: $e');
  }
}
```

Figure 23: fetchEarnings function

This function retrieves the task that accepted by the current user from the 'worker selected task' collection and calculates the total earnings.

```
// Load user data from Firestore based on the current userId
Future<void> _loadUserData() async {
  try {
    DocumentSnapshot userDoc = await FirebaseFirestore.instance
        .collection('Worker Profile') // Updated to match your Firestore collection
        .doc(userId)
        .get();

    if (userDoc.exists) {
      Map<String, dynamic> userData = userDoc.data() as Map<String, dynamic>;
      setState(() {
        usernameController.text = userData['workerName'] ?? '';
        contactController.text = userData['contactNumber'] ?? '';
        profileDetailsController.text = userData['profileDetails'] ?? '';
        locationController.text = userData['location'] ?? '';
        unavailableDateController.text = userData['unavailableDates']?.isNotEmpty ?? false
            ? userData['unavailableDates'][0]
            : '';
      });
    } else {
      print('User data not found.');
    }
  } catch (e) {
    print('Error loading user data: $e');
  }
}
```

Figure 24: _loadUserData

This function retrieves the user data in the collection named 'Worker Profile' by reading the document id and load the specific data in manage profile UI.

```
Future<void> fetchBookingHistory() async {
  if (currentUser == null) return;

  try {
    // Retrieve the tasks from Firestore for the logged-in user
    final snapshot = await FirebaseFirestore.instance
        .collection('worker selected task')
        .where('workerId', isEqualTo: currentUser!.uid)
        .get();

    // Parse the retrieved tasks
    final List<Map<String, dynamic>> bookingList = snapshot.docs.map((doc) {
      final data = doc.data();
      data['id'] = doc.id; // Add the document ID to track the task
      return data;
    }).toList();

    setState(() {
      tasks = bookingList;
    });
  } catch (e) {
    print('Error fetching booking history: $e');
  }
}
```

Figure 25: fetchBookingHistory function

This function retrieves the booking information from the 'worker selected task' collection by reading the workerId as current UID.

```
Future<List<UserCustomer>> _fetchAllUsers() async {
  List<UserCustomer> customers = [];

  // Fetch customers from 'users' collection
  var customerSnapshot = await FirebaseFirestore.instance
      .collection('users')
      .where('role', isEqualTo: 'customer')
      .get();

  for (var doc in customerSnapshot.docs) {
    // Create a UserCustomer object with the document ID and user data
    customers.add(UserCustomer.fromMap(doc.id, doc.data()));
  }

  return customers;
}
```

Figure 26: _fetchAllUsers function

The _fetchAllUsers function retrieves all documents from the users collection in Firestore where the role is set to customer. The information is subsequently assigned to UserCustomer objects for presentation in a ListView.

```
Future<void> _getTerms() async {
  try {
    // Fetch the 'content' field from the 'termsDoc' document in Firestore
    DocumentSnapshot snapshot = await FirebaseFirestore.instance
        .collection('appData')
        .doc('termsDoc')
        .get();

    if (snapshot.exists && snapshot.data() != null) {
      setState(() {···
    } else {
      setState(() {···
    }
  } catch (e) {
    setState(() {···
  }
}

Future<void> _getFaq() async {
  try {
    // Fetch the 'content' field from the 'faqDoc' document in Firestore
    DocumentSnapshot snapshot = await FirebaseFirestore.instance
        .collection('appData')
        .doc('faqDoc')
        .get();

    if (snapshot.exists && snapshot.data() != null) {
      setState(() {···
    } else {
      setState(() {···
    }
  } catch (e) {
    setState(() {···
  }
}
```

Function 27: _getTerms() and _getFaq() function

The _getTerms() and _getFaq() functions retrieve information from the termsDoc and faqDoc documents within the appData collection in Firestore. These files hold the information for the terms and FAQ parts. This is the Read function, during which the application retrieves data from Firestore and presents it in the user interface.

```
Future<void> _saveChanges() async {
  if (_selectedDate != null) {
    try {
      // Format the date to "DD/MM/YY"
      final formattedDate = DateFormat("dd/MM/yy").format(_selectedDate!);

      // Update serviceDate in Firestore using bookingId
      await FirebaseFirestore.instance
          .collection('bookings')
          .doc(widget.bookingId)
          .update({
        'serviceDate': formattedDate,
      });

      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Booking date updated successfully!')),
      );
      Navigator.pop(context);
    } catch (error) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Failed to update booking date: $error')),
      );
    }
  }
```

Figure 28: _saveChanges function

The function _saveChanges is use for saving changes made to “bookings” especially updating the “serviceDate”. It formats the selected date, updates the corresponding document in the bookings collection and provides feedback to the user through snack bars. If have errors, it displays the error message.

```
Future<void> _updateProfile() async {
  try {
    User? currentUser = FirebaseAuth.instance.currentUser;
    if (currentUser != null) {
      Map<String, dynamic> updatedData = {
        'name': _nameController.text,
        'contact': _contactController.text,
      };

      if (_passwordController.text.isNotEmpty) {
        updatedData['password'] = _passwordController.text; // Store plain-text password (not secure)
        await currentUser.updatePassword(_passwordController.text); // Update password in Firebase Auth
      }

      await FirebaseFirestore.instance
          .collection('users')
          .doc(currentUser.uid)
          .update(updatedData);

      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Profile updated successfully!')),
      );
    }
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error updating profile: $e')),
    );
  }
}
```

Figure 29: _updateProfile function

The _updateProfile() function is use to update the user's profile in collection "users" such as name, contact and optionally password and also updates the password in Firebase Authentication. It provides user feedback through SnackBars for successful and error cases.

```
Future<void> updateTaskStatus(int index, String taskId, String bookingId) async {
  try {
    // Update status in 'worker selected task' to "Completed"
    await FirebaseFirestore.instance
        .collection(collectionPath: 'worker selected task')
        .doc(path: taskId)
        .update(data: <Object, Object?>{'status': 'Completed'});

    // Update status in 'bookings' to "Completed"
    await FirebaseFirestore.instance
        .collection(collectionPath: 'bookings')
        .doc(path: bookingId)
        .update(data: <Object, Object?>{'status': 'Completed'});

    // Update status in the selectedTasks list locally
    setState(fn: () {
      selectedTasks[index]['status'] = 'Completed';
    });

    ScaffoldMessenger.of(context: context).showSnackBar(
      snackBar: const SnackBar(content: Text(data: 'Task marked as completed!')),
    );
  } catch (e) {
    ScaffoldMessenger.of(context: context).showSnackBar(
      snackBar: SnackBar(content: Text(data: 'Error updating task status: $e')),
    );
  }
}
```

Figure 30: updateTaskStatus function

This function is to update the status of both ‘worker selected task’ and ‘bookings’ collections to “Completed” in the field named ‘status’.

```
await FirebaseFirestore.instance
    .collection(collectionPath: 'bookings')
    .doc(path: booking['id'])
    .update(data: <Object, Object?>{'status': 'accepted'});
```

Figure 31: part of acceptTask function

This function is to update the status of a booking in the field named ‘status’ to “accepted” in the ‘bookings’ collection.

```
// Update unavailableDates in worker profile
await FirebaseFirestore.instance
    .collection(collectionPath: 'Worker Profile')
    .doc(path: currentUser!.uid)
    .update(data: <Object, Object?>{
  'unavailableDates': FieldValue.arrayUnion(elements: <dynamic>[booking['serviceDate']]),
});
```

Figure 32: part of acceptTask function

This function is to updates the 'unavailableDates' field in the 'Worker Profile' collection for the current worker.

```
Future<void> storeTotalEarnings() async{
  if(currentUser == null) return;

  try{
    //Update total earnings in Firebase
    await FirebaseFirestore.instance
    .collection('Worker Profile')
    .doc(currentUser!.uid)
    .update({'totalIncome': totalEarnings});
  }catch (e){
    print('Error updating total earnings: $e');
  }
}
```

Figure 33: storeTotalEarnings

This function updates the 'totalIncome' field in the 'Worker Profile' collection for the current worker.

```
Future<void> _saveDataLocally() async {
  SharedPreferences prefs = await SharedPreferences.getInstance();
  await prefs.setString('username', usernameController.text);
  await prefs.setString('contact', contactController.text);
  await prefs.setString('profileDetails', profileDetailsController.text);
  await prefs.setString('location', locationController.text);
  await prefs.setString('unavailableDate', unavailableDateController.text);
}
```

Figure 34: _saveDataLocally function

Updates local storage using SharedPreferences to save form data. Although this doesn't update Firestore directly, it updates local data that can later be retrieved or used by the app.

```
// Update profile data including password
Future<void> _updateProfile() async {
  try {
    String userId = widget.initialData['uid'];
    Map<String, dynamic> updatedData = {
      'name': _nameController.text,
      'contact': _contactController.text,
      'password': _passwordController.text, // Adding password to updated data
    };

    await FirebaseFirestore.instance.collection('users').doc(userId).update(updatedData);

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Profile updated successfully!')),
    );
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error updating profile: $e')),
    );
  }
}
```

Figure 35: _updateProfile function

The _updateProfile function modifies the user's record in the Firestore users collection utilizing the document ID (userId). The revised information comprises name, contact, and password.

```
Future<void> _updateTerms() async {
  try {
    // Update the 'content' field in the 'termsDoc' document in Firestore
    await FirebaseFirestore.instance
        .collection('appData')
        .doc('termsDoc')
        .update({'content': _termsController.text});
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Terms updated successfully!')),
    );
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Failed to update terms: $e')),
    );
  }
}

Future<void> _updateFaq() async {
  try {
    // Update the 'content' field in the 'faqDoc' document in Firestore
    await FirebaseFirestore.instance
        .collection('appData')
        .doc('faqDoc')
        .update({'content': _faqController.text});
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('FAQ updated successfully!')),
    );
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Failed to update FAQ: $e')),
    );
  }
}
```

Function 36: _updateTerms and _updateFaq function

The _updateTerms() and _updateFaq() functions serve to refresh the content of the termsDoc and faqDoc documents within Firestore. The material is refreshed with the content from the

TextEditingController fields _termsController and _faqController, accordingly. This represents the Update function, which alters the current terms and FAQ information in Firestore.

2.5.4 Delete

```
Future<void> rejectTask(Map<String, dynamic> booking, int index) async {
  try {
    await FirebaseFirestore.instance.collection(collectionPath: 'bookings').doc(path: booking['id']).delete();

    ScaffoldMessenger.of(context: context).showSnackBar(
      snackBar: const SnackBar(content: Text(data: 'Task rejected.')),
    );
  } catch (e) {
    ScaffoldMessenger.of(context: context).showSnackBar(
      snackBar: SnackBar(content: Text(data: 'Error deleting task: $e')),
    );
  }
}
}
```

Figure 37: rejectTask function

Deletes a booking document from the 'bookings' collection when a task is rejected.

```
// Delete user logic
Future<void> _deleteUser() async {
  try {
    String userId = widget.initialData['uid'];

    await FirebaseFirestore.instance.collection('users').doc(userId).delete();

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('User deleted successfully!')),
    );

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (context) => ManageUsersPage()),
    );
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error deleting user: $e')),
    );
  }
}
```

Figure 38: _deleteUser function

The _deleteUser function removes the user's record from the Firestore users collection. Upon deleting, a success notification appears, and the app returns to the ManageUsersPage.

2.6 Validation

```
TextFormField(
  controller: _emailController,
  decoration: const InputDecoration(
    labelText: 'Email',
    border: OutlineInputBorder(),
  ), // InputDecoration
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Please enter your email';
    }
    final emailRegex = RegExp(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$');
    if (!emailRegex.hasMatch(value)) {
      return 'Enter a valid email address';
    }
    return null;
  },
```

```
TextFormField(
  controller: _passwordController,
  obscureText: true,
  decoration: const InputDecoration(
    labelText: 'Password',
    border: OutlineInputBorder(),
  ), // InputDecoration
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Please enter your password';
    }
    if (value.length < 6) {
      return 'Password must be at least 6 characters';
    }
    return null;
  },
```

Figure 39: Validation for login

There are two parts of validation for our application. The first one is the validation of login. Before enabling the user to log in, the code's validation section makes sure the password and email fields are correctly filled out. The form's state is managed by GlobalKey. Using a regular expression, the email field verifies that the input is not empty and adheres to a proper email format. The password box means that the input is at least six characters long and isn't empty. The login attempt is stopped if either field is incorrect, and the relevant error messages are shown.

```
TextFormField(
  controller: usernameController,
  decoration: const InputDecoration(
    labelText: 'Username',
    border: OutlineInputBorder(),
  ), // InputDecoration
  validator: (value) => value!.isEmpty ? 'Please enter your username' : null,
), // TextFormField
```

```
  labelText: 'Contact Number',
  border: OutlineInputBorder(),
), // InputDecoration
validator: (value) => value!.isEmpty ? 'Please enter your contact number' : null,
), // TextFormField
const SizedBox(height: 26.0),
```

```
labelText: 'Description',
border: OutlineInputBorder(),
), // InputDecoration
validator: (value) => value!.isEmpty ? 'Please enter your description' : null,
), // TextFormField
```

```
labelText: 'Location',
border: OutlineInputBorder(),
), // InputDecoration
validator: (value) => value!.isEmpty ? 'Please enter your location' : null,
), // TextFormField
```

Figure 40: Validation of Manage Profile

The second part of validation is validation of manage profile. When users need to manage their profile, they need to ensure that they input all of the field provided in the page, if not it will appear validation message to inform users to fill in the empty field and unable to save the profile.

```
void _saveBookingDetails() async {
  if (_locationController.text.isEmpty ||
      _dateController.text.isEmpty ||
      selectedPaymentMethod == null) {
    ScaffoldMessenger.of(context: context).showSnackBar(
      snackBar: SnackBar(content: Text(data: "Please fill all fields and select a payment method")),
    );
    return;
  }
```

Figure 41: Validation on Booking

Validate the location field is empty when saving the booking. If empty, it will show the error message.

2.7 Users Permission

```
Future<void> _login() async {
  try {
    // try login
    UserCredential userCredential = await _auth.signInWithEmailAndPassword(
      email: _emailController.text.trim(),
      password: _passwordController.text.trim(),
    );
```

Figure 42: verify for login

This code tries to verify the user through the given email and password. A UserCredential object is returned if the login is successful.

```
// After login, get user role
String uid = userCredential.user!.uid;
DocumentSnapshot userDoc = await _firestore.collection('users').doc(uid).get();
```

Figure 43: get user id for login

The code fetches the uid of the logged-in user. It subsequently queries the users collection in Firestore, retrieving the document identified by the uid to obtain user information, such as the role.

```
      // send user to the page according to their role
      if (role == 'admin') {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (context) => ManageUsersPage()),
        );
      } else if (role == 'customer') {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (context) => MainNavigation(initialTabIndex: 2)),
        );
      } else if (role == 'worker') {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (context) => const WorkerHomeScreen()),
        );
      }
        else {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Unknown User')),
        );
      }
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('User role haven\'t identify')),
      );
    }
  } catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Fail to login: ${e.toString()}')),
    );
  }
}
```

Figure 44: send user according to their role for login

If the userDoc exists and contains a role field, the code validates that the role is an administrator and assigns it to the ManageUsersPage. If the logged in user is a customer it sends it to MainNavigation and assigns the initialTabIndex to 2. If the last role is an employee it navigates directly to the EmployeeHomeScreen. The code also recognizes unrecognized roles and returns a SnackBar with the text “Unrecognized User”. if the userDoc does not exist or is missing a role field, a SnackBar is displayed notifying that the “Unrecognized User Role”. User Role Recognized”.

```
DropdownButtonFormField<String>(
  value: selectedRole,
  items: roles.map((String role) {
    return DropdownMenuItem<String>(
      value: role,
      child: Text(role),
    ); // DropdownMenuItem
  }).toList(),
  onChanged: (newValue) {
    setState(() {
      selectedRole = newValue;
    });
  },
  decoration: const InputDecoration(
    labelText: 'Role',
    border: OutlineInputBorder(),
  ), // InputDecoration
), // DropdownButtonFormField
```

Figure 45: select role for sign up

During registration, the user chooses their role, which sets their permissions post sign-up.

```
if (selectedRole == 'worker') {
  UserWorker userWorker = UserWorker(
    name: nameController.text.trim(),
    email: emailController.text.trim(),
    contact: contactController.text.trim(),
    password: passwordController.text.trim(),
  );
  await _userRepository.createUserWorker(userId, userWorker);
} else {
  UserCustomer userCustomer = UserCustomer(
    name: nameController.text.trim(),
    email: emailController.text.trim(),
    contact: contactController.text.trim(),
    password: passwordController.text.trim(),
  );
  await _userRepository.createUserCustomer(userId, userCustomer);
}
```

Figure 46: create user file for sign up

The type of user (customer or worker) specifies the data model (UserWorker or UserCustomer) utilized and the location of

3.0 User Manual

3.1 Signup/Login page

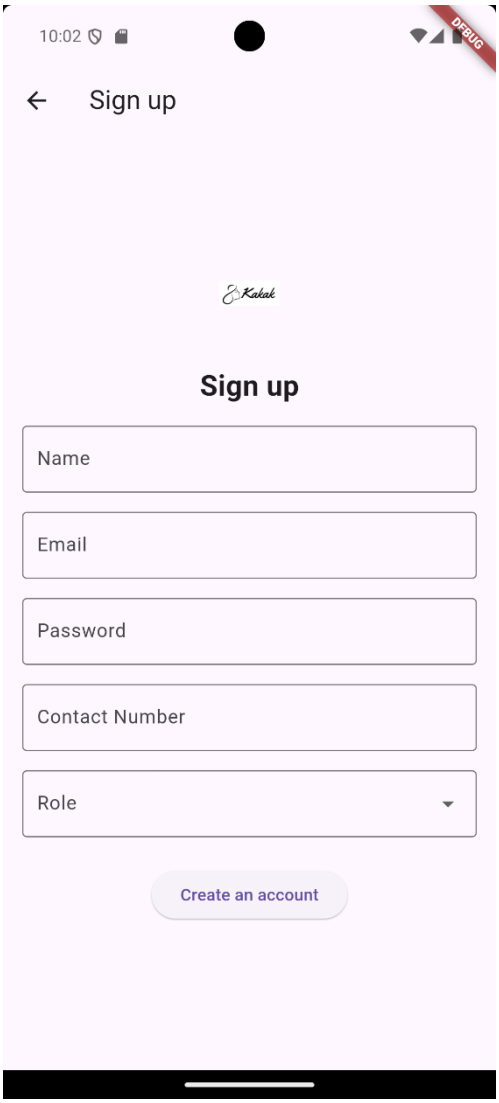
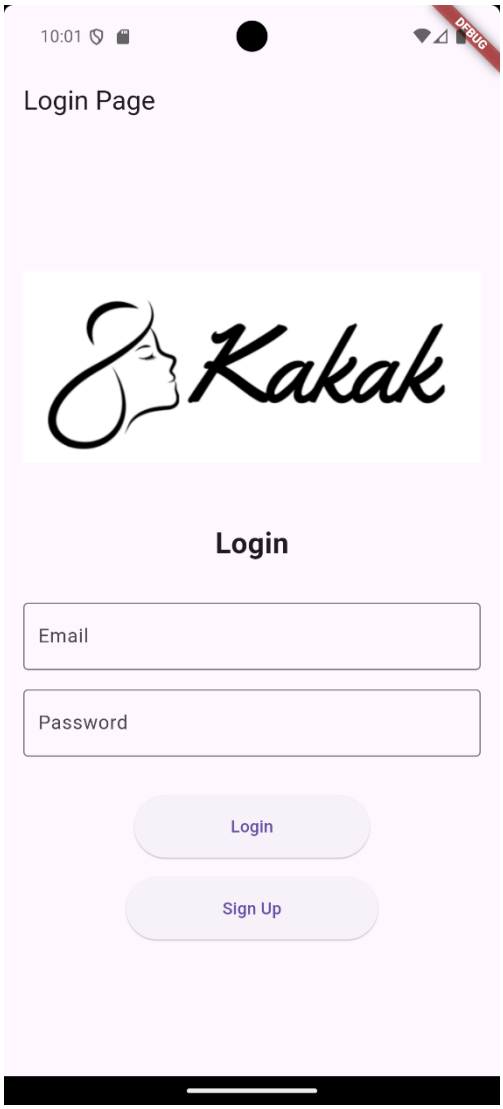


Figure 47 & 48: Sign Up and Login Page



This is the sign-in and sign-up page. The sign-up page lets users create an account by keying in their name, email, password and select their role. This function has validation on user selected role. If the user selects specific roles, it will record the role selected by user and let user login into the specific page. The login page lets the user login into a specific page based on user role.

3.2 Customer home page

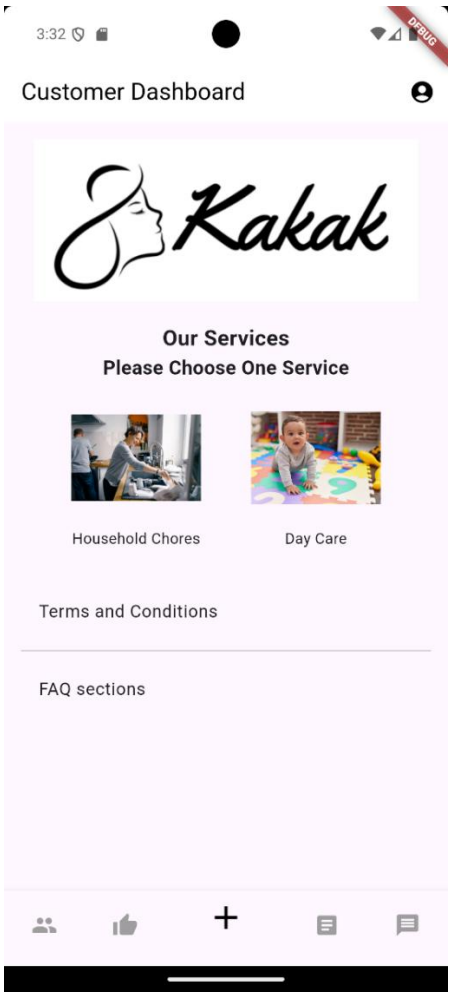
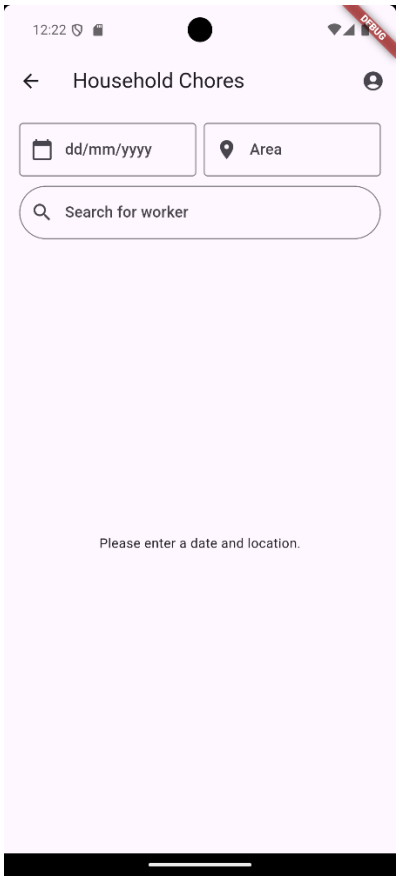


Figure 49: customer home page

The is the user home page for the role “Customer”. In this page, customers can choose one service which are “Household Chores” and “Day Care”. Each of the photo will naviaget to different page based on the selected service. Customers can also see the terms and condition and faq sections.

3.3 Household Chores Service Page and Day Care Service Page



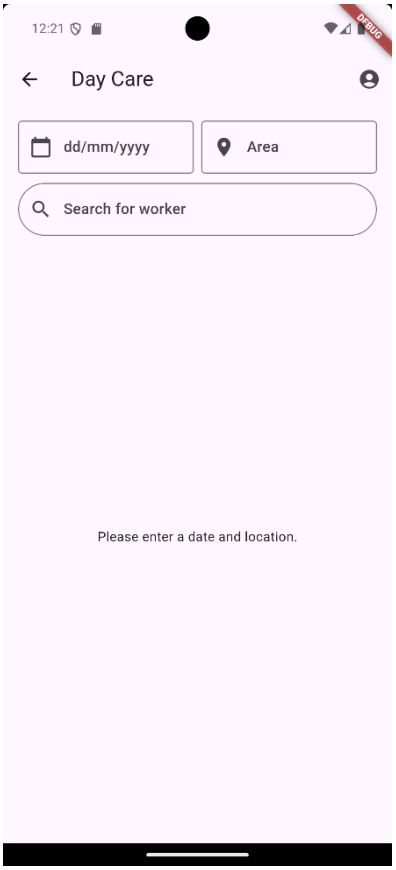


Figure 50, 51: Household Chores Service and Day Care Service pages

This is Household Chores Service Page and Day Care Service Page after clicking on the photo based on different photo. Customers need to input the date and area that want the service. It will show the list to customers that which workers are available on that date and area.

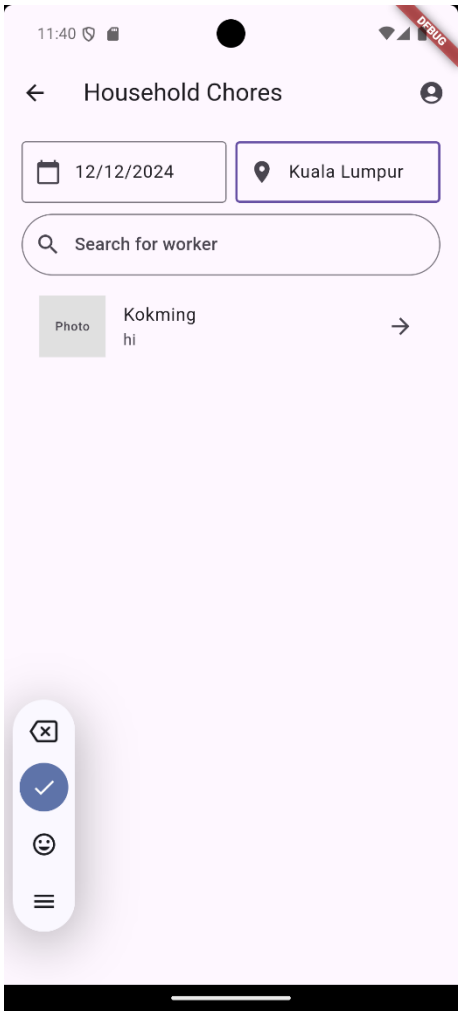


Figure 52: HouseHold Chores page

This is Household Chores Page. After inputting the date and area, the available worker will display on the list. Customer can choose the worker they want. There are same way to use with Household Chores Service Page and Day Care Service Page.

3.4 Worker Details Page

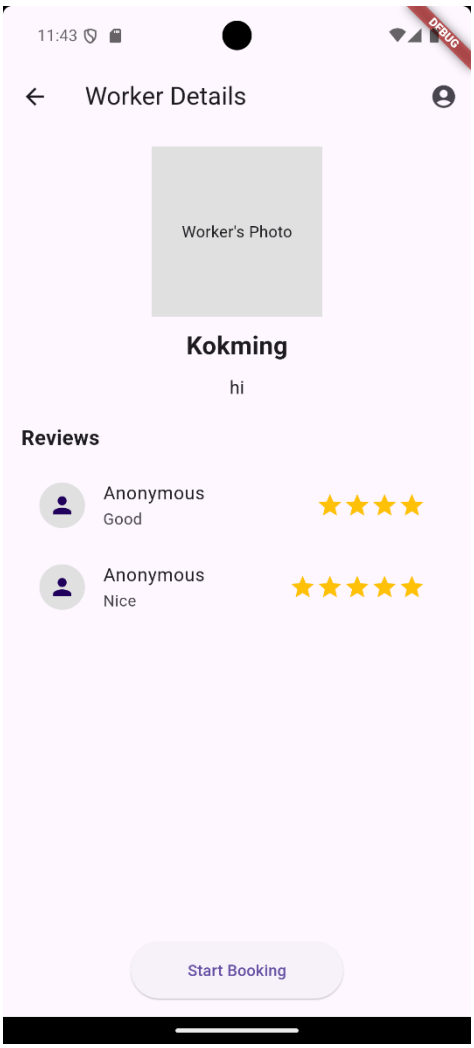


Figure 53: Worker Details Page

This is Worker Details Page. After choosing the worker from the list, customers can see the review and rating from other customers. Customers can see the selected worker's review and rating from other customers. Then customers can start the bookings with clicking "Start Booking".

3.5 Confirm Booking Page

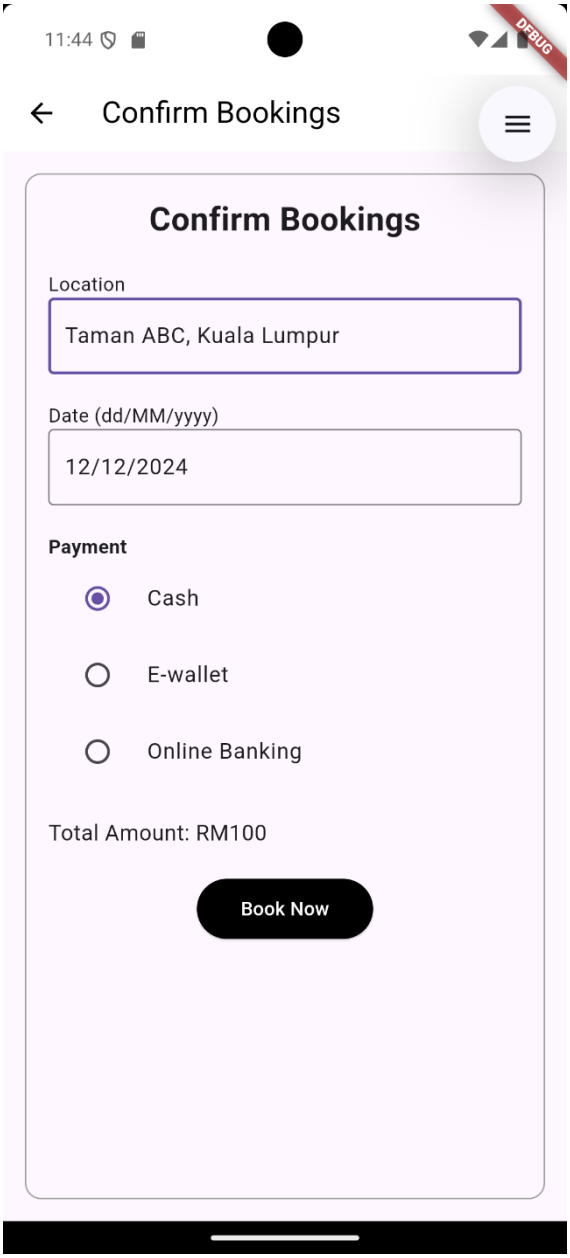


Figure 54: Confirm Booking Page

This is Confirm Booking Page. Customer will go to Confirm Booking Page after clicking "Start Booking". Customer need to input the location and date to book the worker. After that click "Book Now" button, the booking had been book.

3.6 Our Service Page

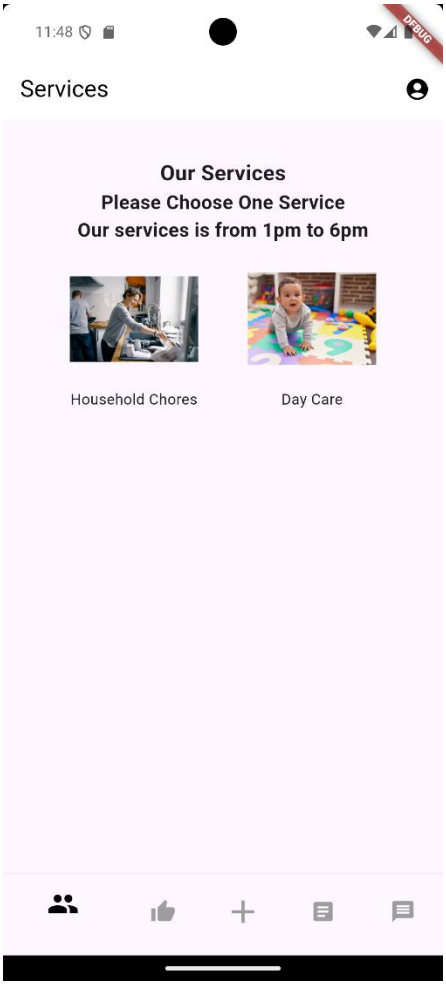


Figure 55: Our Service Page

Our Service Page used to show what service that have to customer, same with Customer Home Page, when customer choose the service, it will go to different service page such as Household Chores Service Page and Day Care Service Page.

3.7 Booking History Page

Figure 56: Booking History Page

This is Booking History Page. Customer can check the Booking History with this page.

3.8 Booking Details Page

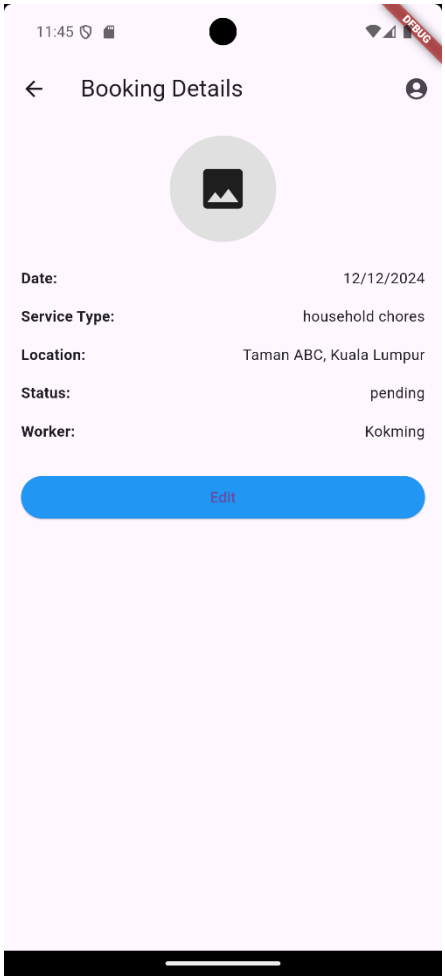


Figure 57: Booking Details Page

This page is Booking Details Page. Customer can click the booking from Booking History Page and go to Booking Details Page to see the booking details.

3.9 Edit Booking Date Page

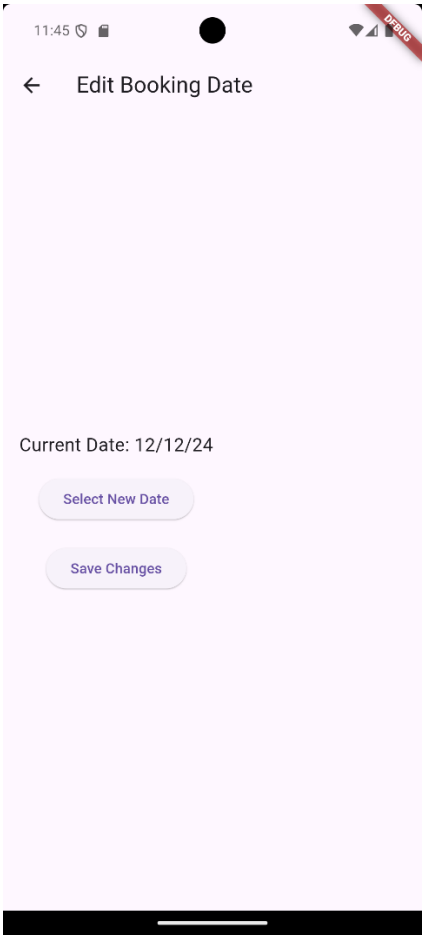


Figure 58: Edit Booking Date Page

This page is Edit Booking Page. Customer can edit the booking time when the status is "pending". After worker accept the booking, it cannot to edit anymore.

3.10 Review and Rating for Worker Page

Figure 59: Review and Rating Worker Page

This page is Review and Rating for Worker Page. After the worker has completed their job, the status of bookings will become "completed". So, customers can give a review and rating after worker had completed their job.

3.11 Give Review and Rating for Worker Page

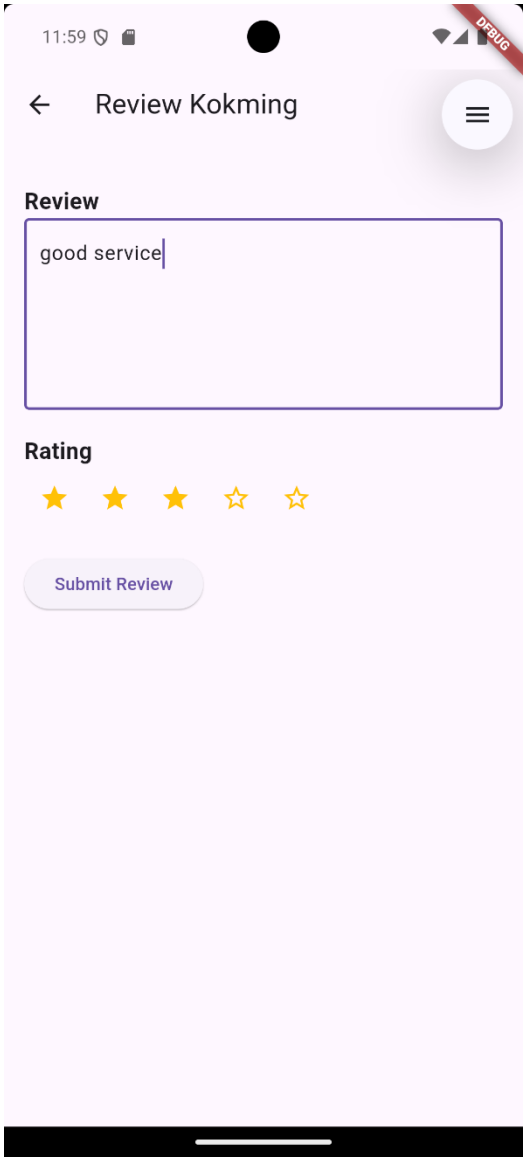


Figure 60: Give Review and Rating Page

This is Give Review and Rating for Worker Page. After clicking the worker that want to give review, customer can give review and rating to the worker.

3.12 Chat with Admin Page

Figure 61: Chat With Admin Page

This is Chat with Admin Page. When customers face problem or need to complain about worker can use this page to contact with admin.

3.13 Customer Profile Page

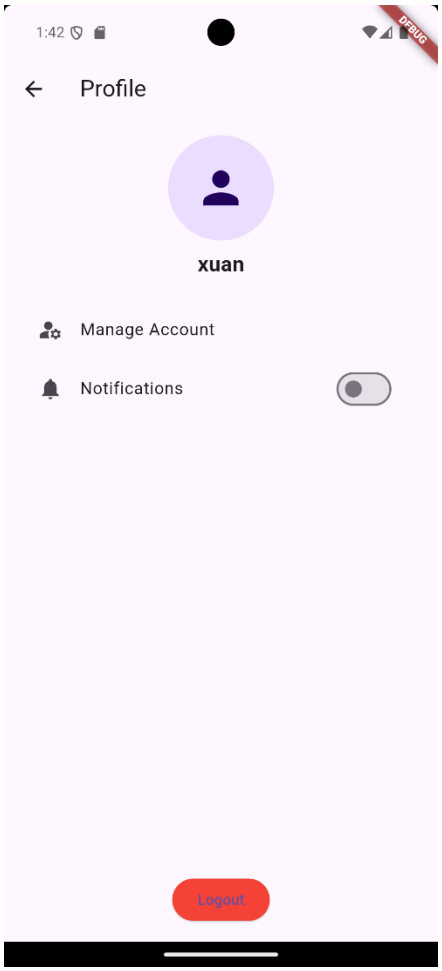


Figure 62: Customer Profile Page

This is Customer Profile Page. When clicking the right-hand side profile icon, it will go to Customer Profile Page. Customer can see their name under the profile icon.

3.14 Edit Customer Profile Page

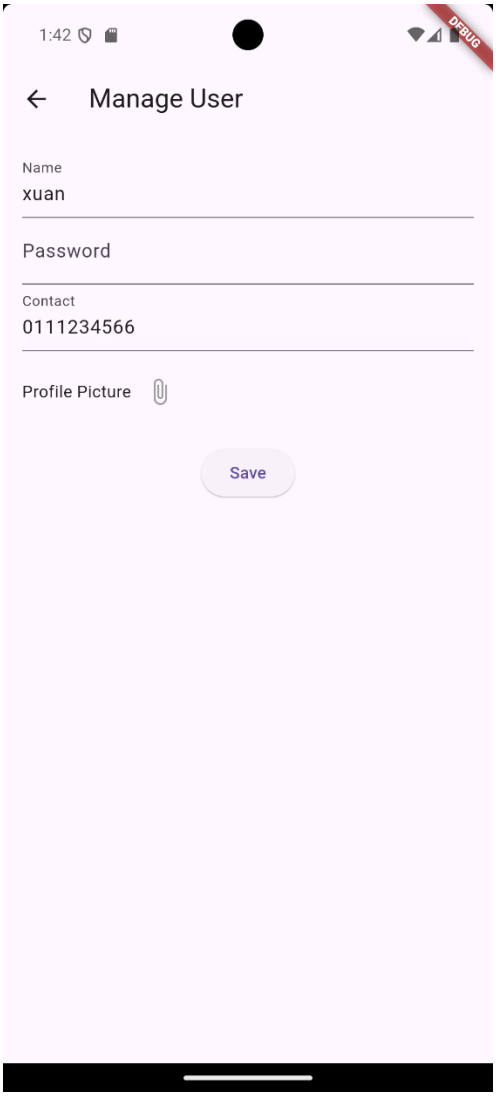


Figure 63: Edit Customer Profile Page

This is Edit Customer Profile Page. Customer can edit their name, password and contact.

3.15 Woker Home Page

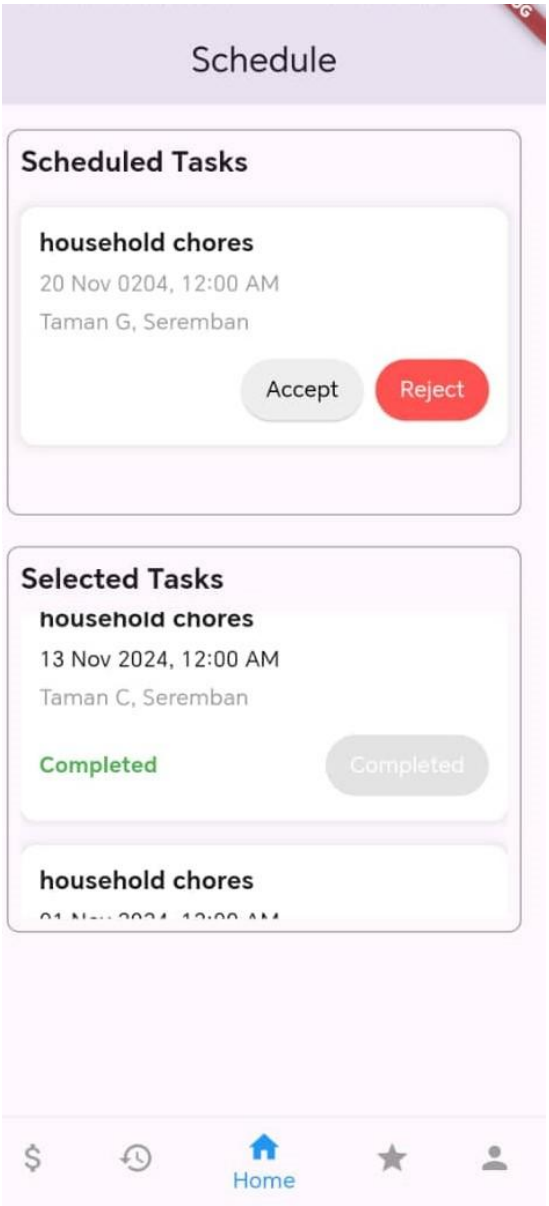


Figure 64: Worker Home Page

In the worker home page, worker will be able to accept or reject the task in the 'Scheduled Task' that customer assigned or booked to them. If the worker accepts the task, the task will then show in the 'Selected Task'. Workers can change the status as completed by clicking the 'Mark as completed' button.

3.16 Worker Profile Page

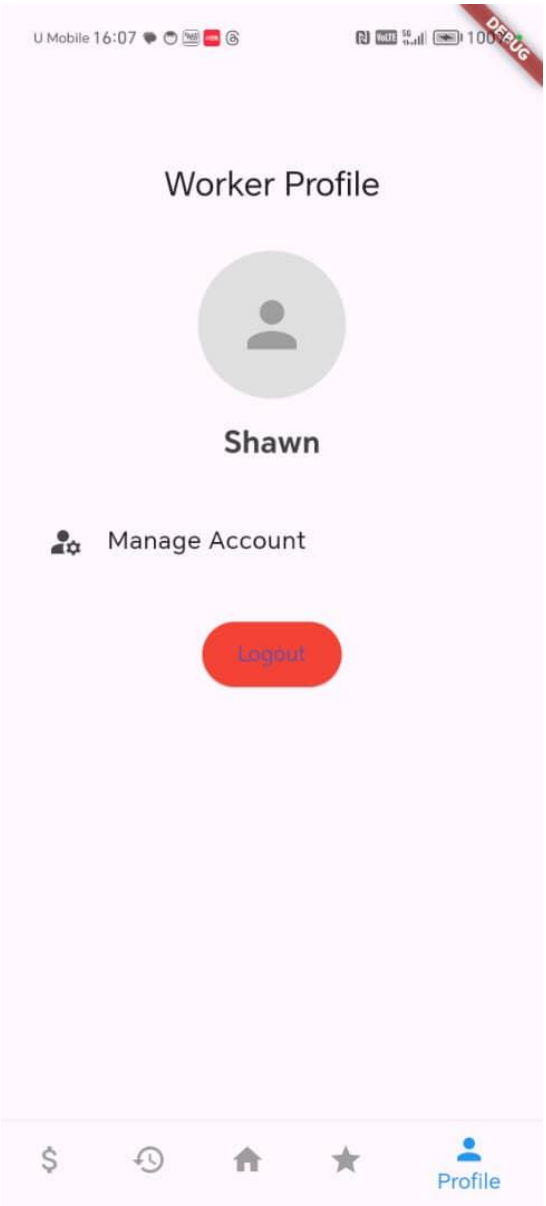


Figure 65: Worker Profile Page

On the worker profile page, the username will show up under the profile picture icon. Workers can click the manage account to navigate to the 'Manage Account Page'. Worker can also change the status of the notification. The Logout button will navigate to the login page.

3.17 Manage Profile Page

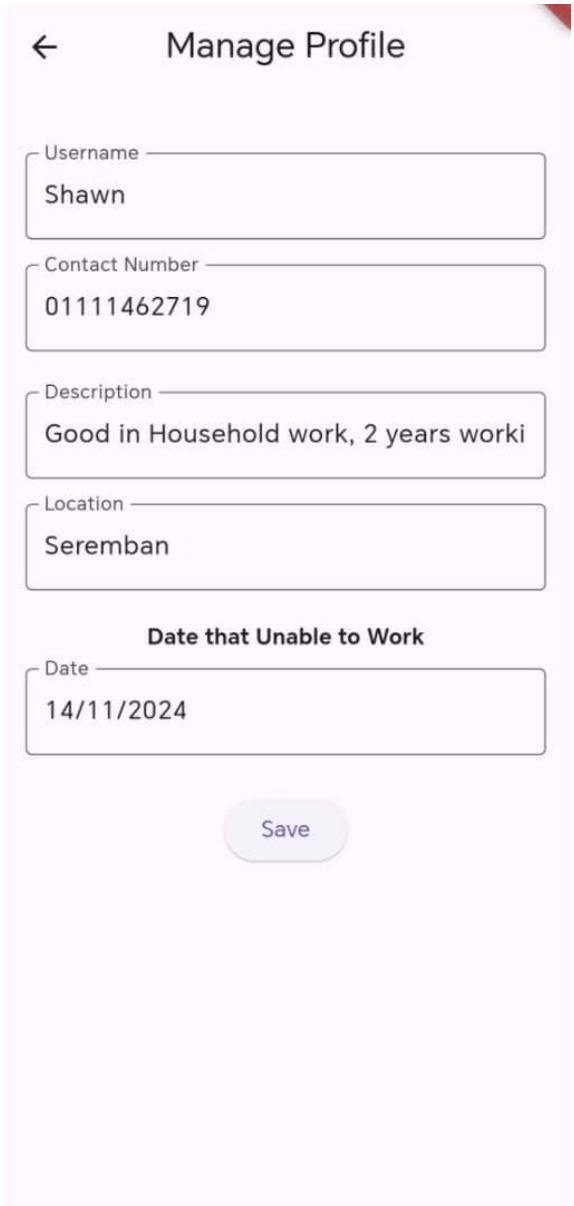


Figure 66: Manage Account Page

Workers can update their information in the specific field. After click the save button, the information will been saved successfully.

3.18 Review and Rating Page

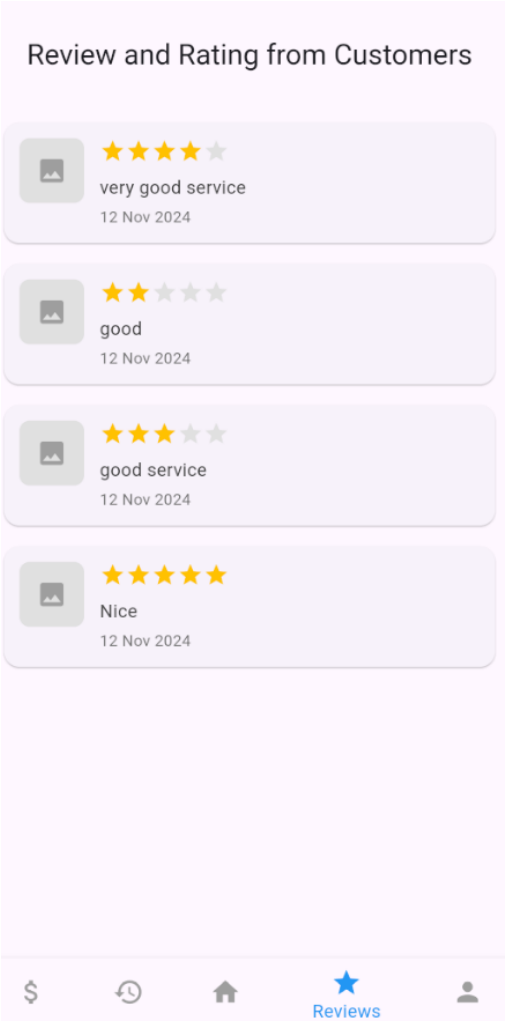


Figure 67: Reviews and Rating Page

Workers can view the review and rating in this page.

3.19 Task History Page

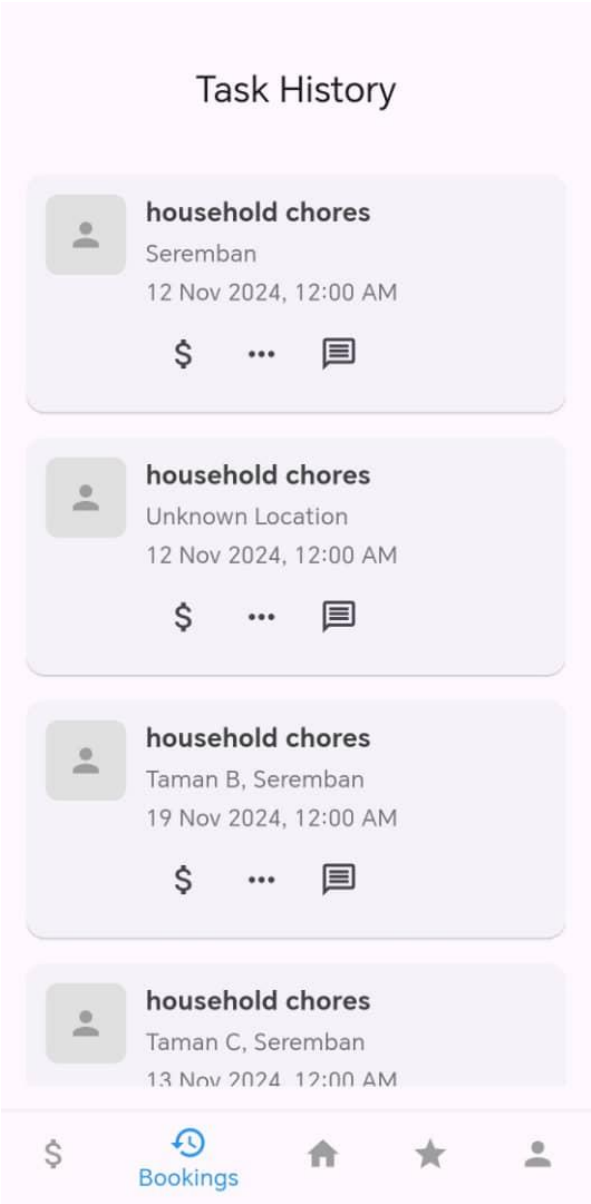


Figure 68: Task History Page

Workers can view the tasks that they accepted on the Task History Page.

3.20 Money Earned Page

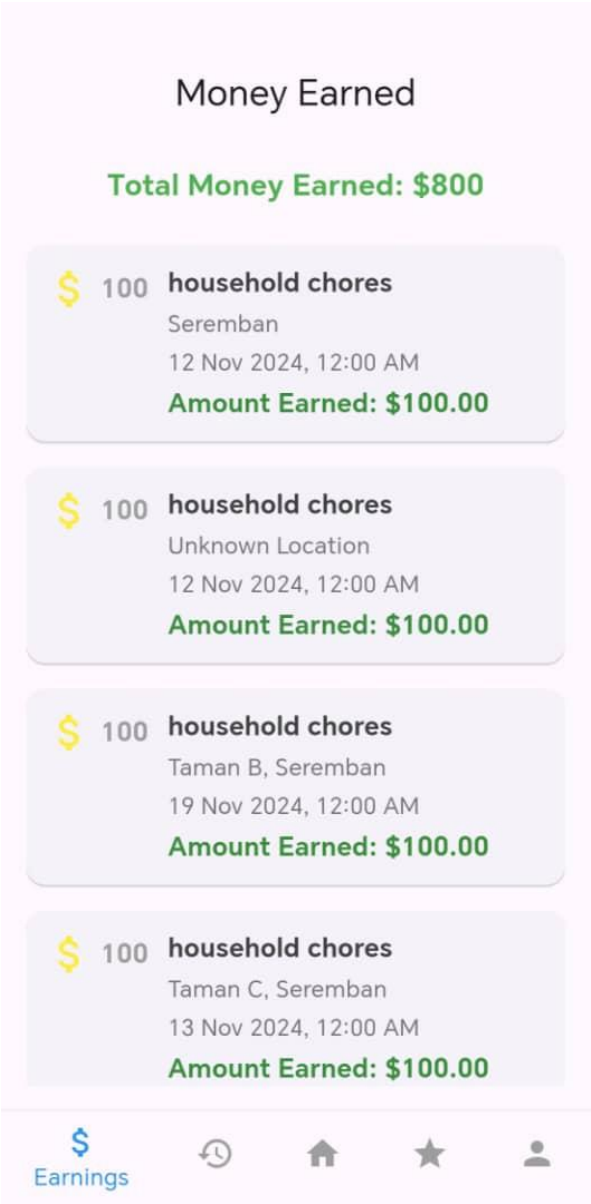


Figure 69: Money Earned Page

When workers complete the task, the the money earned page will show the earned money and the total money earned.

3.21 Manage User Page

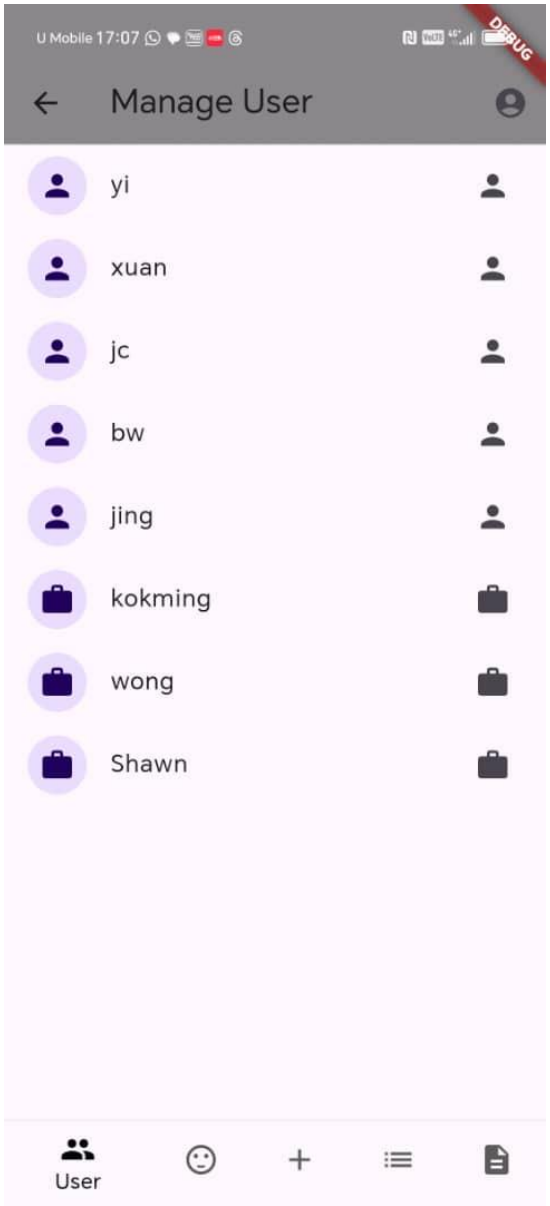


Figure 70: Manage User Page

Whenever an administrator logs in, they are automatically redirected to this page, which displays all Sign Up users and puts icons to the right of each user to differentiate between customers and workers. Administrators can click on the user to make changes to that user's account.

3.22 Manage Complain Page

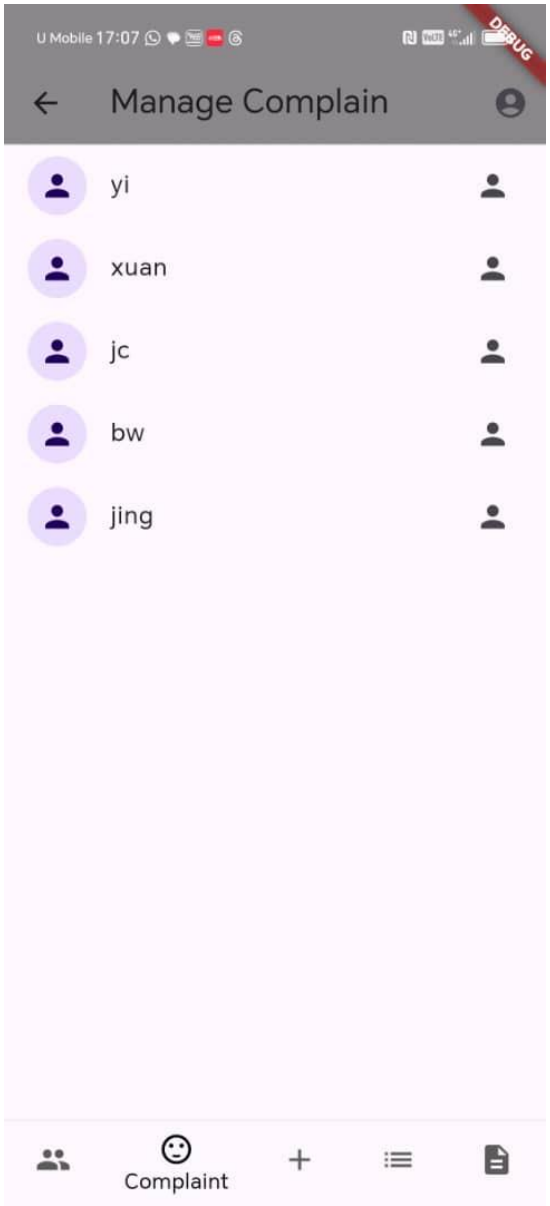


Figure 71: Manage Complain Page

This page is used to communicate with users. Administrators can click on the user in order to talk to the user.

3.23 Edit Content Page

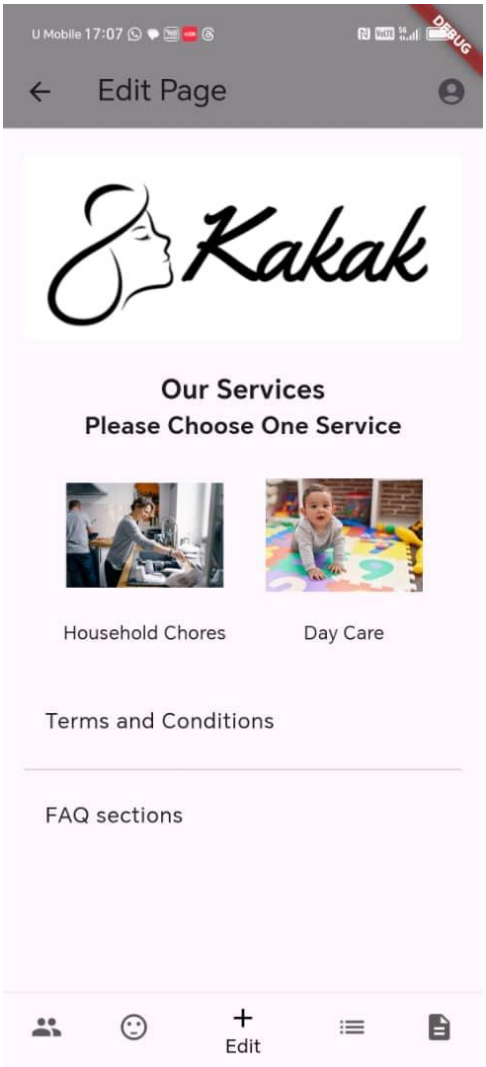


Figure 72: Edit Page

On this page, administrators can view the client's dashboard and make changes by clicking on the terms and conditions and FAQ sections below to jump to the corresponding page.

3.24 Refund Page

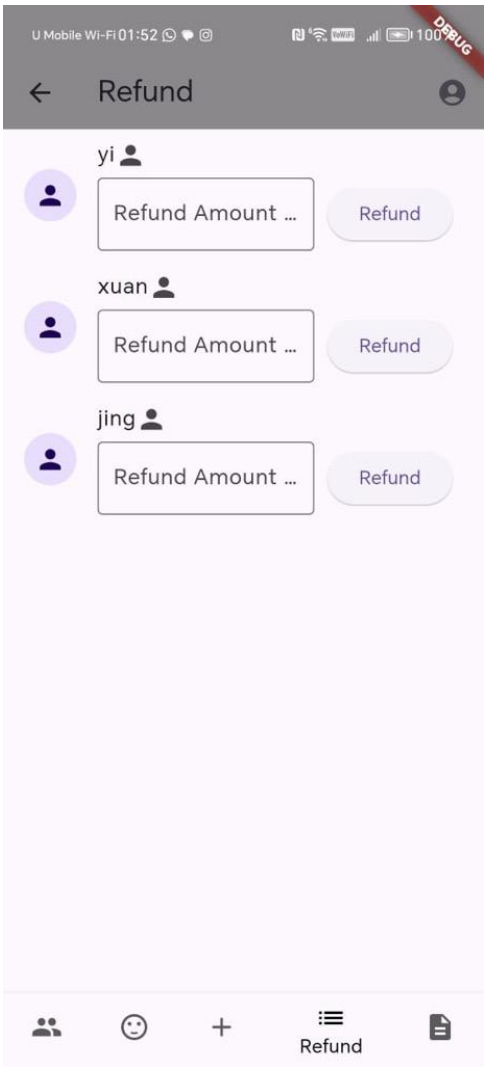


Figure 73: Refund Page

The purpose of the Refund page is for refunding fees to customers, the administrator can enter the amount to be refunded in the box and click on the REFUND button to complete the refund action, while clicking on it a pop-up window will appear to show that the refund has been made and for what amount.

3.25 Report Page

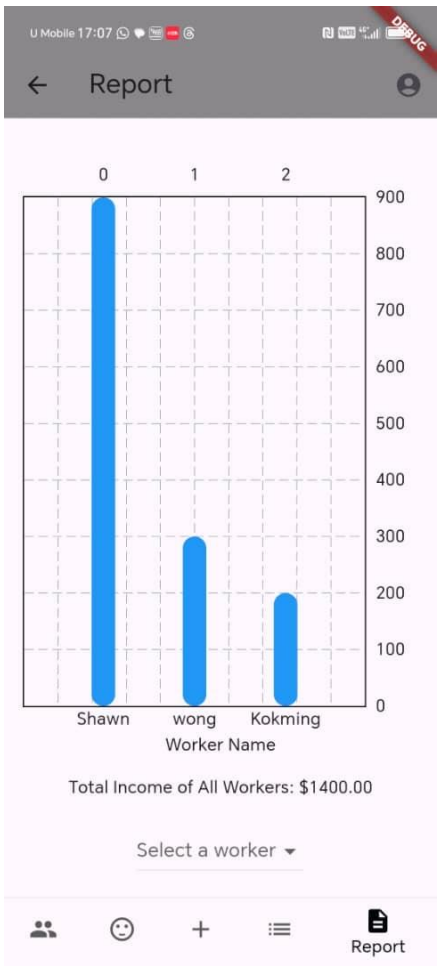


Figure 74: Report Page

The report page will show the total income of the highest five staff members in the chart, and will show the total income of all staff members in the chart under the devolution, and administrators can also click on the selected box below to select the staff to view the staff can be informed of the total income of the staff.

3.26 Manage User Profile Page

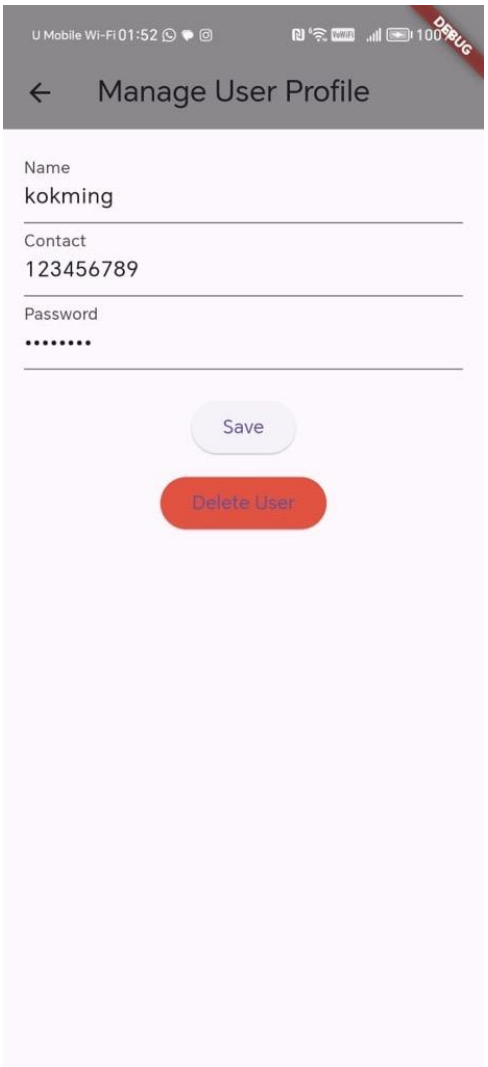


Figure 75: Manage User Profile Page

The page is used to manage the user's account, when the administrator use the manage user page and click on the user will jump to the page, the page will fill in the box to enter the user's information, the administrator can be adjusted to the information, and by clicking on the save button can be updated to update the user's information, and the administrator can also be clicked on delete user to delete the user.

3.27 Chat Page

Figure 76: Chat Page

This page is used to communicate with the customer's page, when the administrator uses the explain page to click on the user will jump to the page, the page will show the previous conversation with the customer, the administrator can enter the information in the input box, and then click on the send button to complete the delivery of information.

3.28 Terms and Condition/FAQ Page

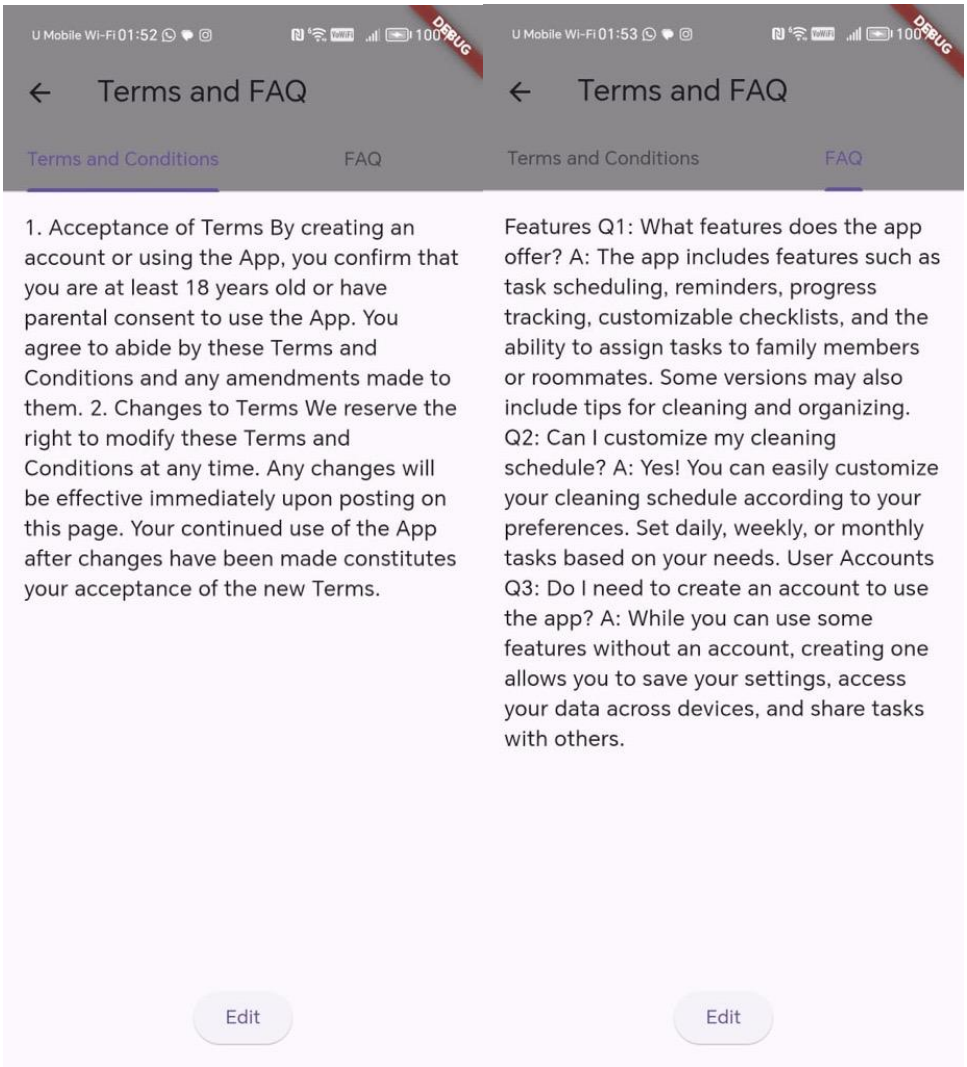


Figure 77, 78: Terms and Condition/ FAQ Page

This page is used to modify the information of the page, when the administrator uses the edit page to click on the terms and condition or FAQ section will jump to this page, the page will show the detailed information, if the administrator wants to change can be changed through the bottom of the Edit button to change the information.

3.29 Admin Profile Page

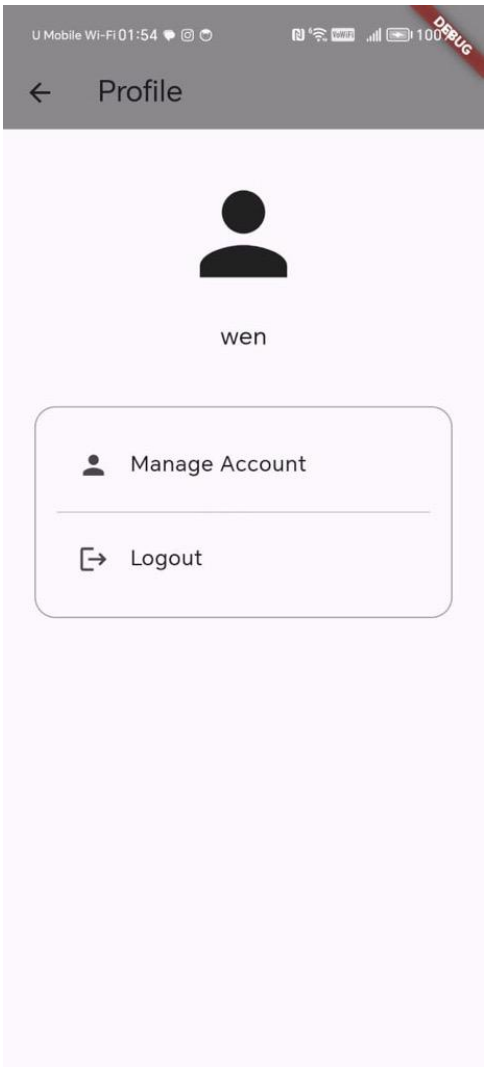


Figure 79: Admin Profile Page

This page is used to view the administrator's account, the page will display the administrator's name, and there are Manage Accout and Logout options, click on the former to jump to the Manga User Profile Page to change the user's information, if you use the latter to log out of the account.

4.0 Automated System Testing

Automated testing of systems is crucial in modern software development, enabling teams to deliver high-quality products efficiently. Utilizing specialized tools to automate test execution can reduce manual labor and the chances of human mistakes, leading to faster and more reliable testing. Automation enables thorough regression, functional, and performance testing, improving both coverage and consistency. It additionally promotes ongoing integration and deployment, speeding up development and ensuring system dependability. In conclusion, automated testing improves product quality, reduces time-to-market, and is vital in the software development lifecycle.

In this automated testing procedure, we utilized Firebase Test Lab as our main platform for automation, which also functions as our data storage option. Firebase Test Lab provides robust testing tools that can execute automated tests on diverse devices and environments to guarantee the application's stability and compatibility. By using Test Lab, we can replicate various user scenarios and device setups, enabling us to detect possible problems and improve the app's overall quality.

Figure 80: Screen shot of the test lab successfully

The Test Lab environment enables comprehensive analysis, presenting clear results for each automated test. From the provided images, we can observe that the testing process by the automated system concluded successfully, with a total duration of 3 minutes and 31 seconds. Throughout this period, the system activated 76 interactions, demonstrating the thoroughness and coverage of the test cycle.

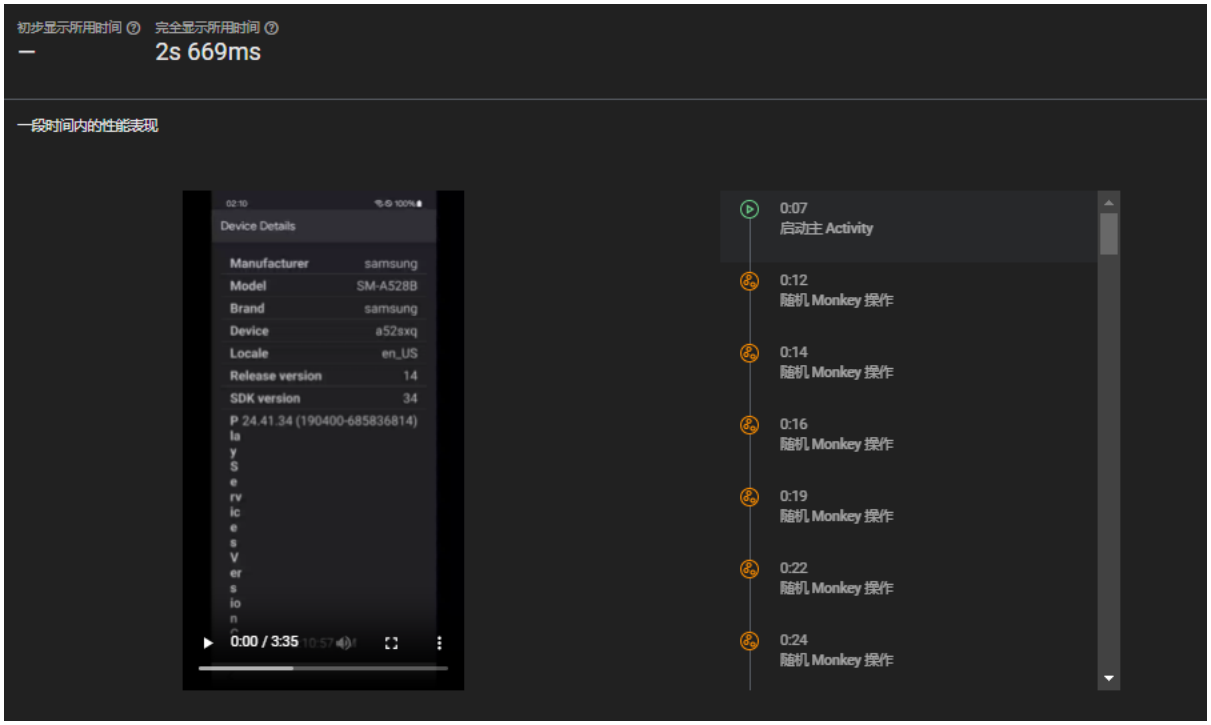


Figure 81: Screen shot of the testing record

One of the featured images displays the test video for the login and registration screens. The video documents the entire testing procedure, outlining the stages and results. Every interaction, starting from the entry of credentials to the final confirmation of successful account creation or login, is documented and examined to guarantee the process operates as intended. The test ends when all designated scenarios are confirmed.

5.0 Conclusion

In conclusion, the "Kakak" software provides a much-needed remedy for the difficulties contemporary families encounter in handling their domestic duties. Through its capacity to match users with qualified domestic helpers, the app offers a smooth platform that guarantees cost, convenience, and confidence. "Kakak" promotes openness and comfort for both clients and employees with features including background checks, reviews, and an intuitive user interface. The app's all-encompassing strategy promises a dependable, productive, and economical solution for household administration, even in the face of obstacles like labour availability and service consistency. In the end, "Kakak" positions itself as the go-to platform for a smarter, more manageable house by satisfying the demands of busy families while simultaneously providing domestic workers with worthwhile earning options